

Classification Model Training Notebook

Setup Environment

```
In [78]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipynrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

```
In [ ]: # <Student to fill this section>
group_name = "36106-26AU-AT3-Group 24"
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

```
In [81]: # Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Nana Ama Goldwater

```
In [82]: # Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

26137455

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: !

```
pip install <package_name>
```

```
In [83]: # No additional packages required.  
# scikit-learn, pandas, numpy, altair are pre-installed in Colab.
```

0.b Import Packages

```
In [84]: import re  
import numpy as np  
import pandas as pd  
import altair as alt  
  
from sklearn.model_selection import train_test_split  
from sklearn.impute import SimpleImputer  
from sklearn.preprocessing import StandardScaler  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.dummy import DummyClassifier  
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score,  
    roc_auc_score, average_precision_score,  
    confusion_matrix, classification_report,  
)  
  
alt.data_transformers.disable_max_rows()  
pd.set_option('display.max_columns', None)  
pd.set_option('display.width', 200)  
RANDOM_STATE = 42
```

B. Business Understanding

```
In [85]: business_use_case_description = """  
Use case: Repeat-purchase prediction for a global online/offline retailer.  
  
The retailer has accumulated three years of transactional data (~31k orders  
from ~19k customers) and wants to identify, at any point in time, which of  
their existing customers are most likely to place an order in the next 90  
days. The goal is to enable proactive customer engagement: marketing spend,  
loyalty offers, and sales-rep attention can be concentrated on the customers  
most likely to convert in the near term.  
  
This is a binary classification problem:  
- Each row represents one customer at a fixed cutoff date.  
- The target is 1 if the customer placed at least one order in the 90 days  
  AFTER the cutoff, 0 otherwise.  
- Features are aggregated from the customer's order history STRICTLY  
  BEFORE the cutoff, so the model can be applied in production by  
  setting the cutoff to 'today'.  
"""
```

```
In [86]: # Do not modify this code  
print_tile(size="h3", key='business_use_case_description', value=business_use_case_description)
```

business_use_case_description

Use case: Repeat-purchase prediction for a global online/offline retailer. The retailer has accumulated three years of transactional data (~31k orders from ~19k customers) and wants to identify, at any point in time, which of their existing customers are most likely to place an order in the next 90 days. The goal is to enable proactive customer engagement: marketing spend, loyalty offers, and sales-rep attention can be concentrated on the customers most likely to convert in the near term. This is a binary classification problem: - Each row represents one customer at a fixed cutoff date. - The target is 1 if the customer placed at least one order in the 90 days AFTER the cutoff, 0 otherwise. - Features are aggregated from the customer's order history STRICTLY BEFORE the cutoff, so the model can be applied in production by setting the cutoff to 'today'.

```
In [87]: business_objectives = """
Business impact of correct vs incorrect predictions:

True positive (predicted reorder; customer did reorder):
- Targeted offer reaches a high-intent customer. Marketing spend is
  efficient; direct uplift to revenue retention.

False positive (predicted reorder; customer did not reorder):
- Wasted cost per contact (email + offer). Bounded and RECOVERABLE.

True negative (predicted no reorder; customer did not reorder):
- No campaign spend, no missed opportunity neutral outcome.

False negative (predicted no reorder; customer would have reordered):
- High-value customer not contacted; may churn naturally. Direct loss
  of LTV contribution. NOT directly recoverable. Re-acquisition costs
  far more than retention.

Asymmetric cost structure → recall on the positive class is more important
than precision. The model should err toward FLAGGING borderline customers.
This drives the metric and threshold choices in Section J.
"""
```

```
In [88]: # Do not modify this code
print_tile(size="h3", key='business_objectives', value=business_objectives)
```

business_objectives

Business impact of correct vs incorrect predictions: True positive (predicted reorder; customer did reorder): - Targeted offer reaches a high-intent customer. Marketing spend is efficient; direct uplift to revenue retention. False positive (predicted reorder; customer did not reorder): - Wasted cost per contact (email + offer). Bounded and RECOVERABLE. True negative (predicted no reorder; customer did not reorder): - No campaign spend, no missed opportunity neutral outcome. False negative (predicted no reorder; customer would have reordered): - High-value customer not contacted; may churn naturally. Direct loss of LTV contribution. NOT directly recoverable. Re-acquisition costs far more than retention. Asymmetric cost structure → recall on the positive class is more important than precision. The model should err toward FLAGGING borderline customers. This drives the metric and threshold choices in Section J.

```
In [89]: stakeholders_expectations_explanations = """
Users of the predictions:
- Marketing team: receives a ranked customer list each month and decides
  which segments receive which campaigns. They use the PROBABILITY
  output (not the hard 0/1 label) to size campaigns to their budget.
- Sales reps: for business / store accounts (12% offline channel), the
  score flags accounts that warrant a proactive call.
- Finance / planning: aggregate predicted reorder counts feed short-term
  revenue forecasts.

Stakeholders impacted by the predictions:
- Customers flagged as 'likely reorder' receive more communication;
  fairness check needed to ensure no protected segment is systematically
  over- or under-targeted.
- Customers flagged as 'unlikely reorder' receive less communication;
  a biased model could accelerate churn in early-tenure segments.

Deliverable:
- A probability score per customer, refreshed monthly.
- A documented threshold tied to campaign cost and expected LTV uplift.
- Clear interpretability: which features drove each score, so marketing
  can sanity-check the model before acting on it.
"""
```

```
In [90]: # Do not modify this code
print_tile(size="h3", key='stakeholders_expectations_explanations', value=stakeholders_expectations_explanations)
```

Users of the predictions: - Marketing team: receives a ranked customer list each month and decides which segments receive which campaigns. They use the PROBABILITY output (not the hard 0/1 label) to size campaigns to their budget. - Sales reps: for business / store accounts (12% offline channel), the score flags accounts that warrant a proactive call. - Finance / planning: aggregate predicted reorder counts feed short-term revenue forecasts. Stakeholders impacted by the predictions: - Customers flagged as 'likely reorder' receive more communication; fairness check needed to ensure no protected segment is systematically over- or under-targeted. - Customers flagged as 'unlikely reorder' receive less communication; a biased model could accelerate churn in early-tenure segments. Deliverable: - A probability score per customer, refreshed monthly. - A documented threshold tied to campaign cost and expected LTV uplift. - Clear interpretability: which features drove each score, so marketing can sanity-check the model before acting on it.

C. Data Understanding

C.1 Load Datasets

```
In [91]: import os

# Reload raw source tables. This notebook is self-contained.
try:
    customer_df = pd.read_csv('/content/customer.csv')
    sales_order_header_df = pd.read_csv('/content/sales_order_header.csv')
except Exception as e:
    print(e)

print(f'customer.csv           : {customer_df.shape[0]:,} rows x {customer_df.shape[1]} cols')
print(f'sales_order_header.csv : {sales_order_header_df.shape[0]:,} rows x {sales_order_header_df.shape[1]} cols')

def to_snake_case(name):
    s = name.strip().replace(' ', '_')
    s = re.sub(r'([A-Z][a-z]+)', r'\1_\2', s)
    s = re.sub(r'([a-z0-9])([A-Z])', r'\1_\2', s)
    return s.lower()

cust = customer_df.copy();      cust.columns = [to_snake_case(c) for c in cust.columns]
sales = sales_order_header_df.copy(); sales.columns = [to_snake_case(c) for c in sales.columns]

customer.csv           : 14,275 rows x 5 cols
sales_order_header.csv : 31,465 rows x 17 cols
```

C.2 Define Target variable

```
In [92]: # Convert order_date to datetime; identify global cutoff.
sales['order_date'] = pd.to_datetime(sales['order_date'], errors='coerce')
sales = sales.dropna(subset=['order_date', 'customer_id']).reset_index(drop=True)

PREDICTION_WINDOW_DAYS = 90
max_date = sales['order_date'].max()
cutoff_date = max_date - pd.Timedelta(days=PREDICTION_WINDOW_DAYS)

print(f'Total orders           : {len(sales):,}')
print(f'Date range              : {sales["order_date"].min().date()} → {max_date.date()}')
print(f'Cutoff (features ≤)       : {cutoff_date.date()}')
print(f'Label window from         : {(cutoff_date + pd.Timedelta(days=1)).date()} → {max_date.date()}')

Total orders           : 31,465
Date range             : 2011-05-30 → 2014-06-29
Cutoff (features ≤)    : 2014-03-31
Label window from      : 2014-04-01 → 2014-06-29
```

In [92]:

```
In [93]: target_definition_explanations = """
Target variable: `will_reorder` ∈ {0, 1}.

Definition (per customer):
- 1 if the customer placed at least one order in (cutoff_date, max_date].
- 0 otherwise.

Cutoff choice: max(order_date) - 90 days.
- 90 days matches a typical quarterly retention-campaign cadence.
- With ~1.65 orders/customer over 3 years, a shorter window would yield
  too few positives; a longer one would slow campaign feedback loops.

Eligibility: only customers with ≥1 order BEFORE the cutoff are included.
Customers whose first appearance is after the cutoff have no feature
history and cannot be scored.

Binary (not regression): the downstream decision (campaign yes/no) is
itself binary, so the model output should match the decision granularity.
"""

In [94]: # Do not modify this code
print_tile(size="h3", key='target_definition_explanations', value=target_definition_explanations)

target_definition_explanations
```

Target variable: `will\_reorder` ∈ {0, 1}. Definition (per customer): - 1 if the customer placed at least one order in (cutoff_date, max_date]. - 0 otherwise. Cutoff choice: max(order_date) - 90 days. - 90 days matches a typical quarterly retention-campaign cadence. - With ~1.65 orders/customer over 3 years, a shorter window would yield too few positives; a longer one would slow campaign feedback loops. Eligibility: only customers with ≥1 order BEFORE the cutoff are included. Customers whose first appearance is after the cutoff have no feature history and cannot be scored. Binary (not regression): the downstream decision (campaign yes/no) is itself binary, so the model output should match the decision granularity.

C.3 Create Target variable

```
In [95]: pre = sales[sales['order_date'] <= cutoff_date].copy()
post = sales[sales['order_date'] > cutoff_date].copy()

eligible_ids = pre['customer_id'].unique()
reorderer_ids = set(post['customer_id'].unique())

target_df = pd.DataFrame({'customer_id': eligible_ids})
target_df['will_reorder'] = target_df['customer_id'].isin(reorderer_ids).astype(int)
target_name = 'will_reorder'

print(f'Eligible customers (≥1 pre-cutoff order): {len(target_df):,}')
print(f'Positive class : {target_df[target_name].sum():,} ({target_df[target_name].mean():.2%})')
print(f'Negative class : {(target_df[target_name]==0).sum():,}')

Eligible customers (≥1 pre-cutoff order): 16,300
Positive class : 2,240 (13.74%)
Negative class : 14,060
```

C.4 Explore Target variable

```
In [96]: balance = (target_df[target_name].value_counts()
               .rename_axis('class').reset_index(name='count'))
balance['proportion'] = balance['count'] / balance['count'].sum()
balance['label'] = balance['class'].map({0: 'no reorder', 1: 'reorder'})
display(balance.style.format({'proportion': '{:.2%}'))
```

	class	count	proportion	label
0	0	14060	86.26%	no reorder
1	1	2240	13.74%	reorder

```
In [97]: # Visualise class balance + distribution of reorder counts per positive customer
```

```
reorder_counts = (post.groupby('customer_id').size()
                  .rename('n_reorders').reset_index())
print('Reorder-window orders per positive customer:')
print(reorder_counts['n_reorders'].describe().round(2))

alt.Chart(balance).mark_bar().encode(
    x=alt.X('label:N', title='Class'),
    y=alt.Y('count:Q', title='Customers'),
    color=alt.Color('label:N', legend=None),
    tooltip=['label', 'count', alt.Tooltip('proportion:Q', format='.2%')]
).properties(width=320, height=240, title='Target class balance')
```

Reorder-window orders per positive customer:

```
count    5059.00
mean       1.07
std        0.41
min        1.00
25%        1.00
50%        1.00
75%        1.00
max        13.00
```

Name: n_reorders, dtype: float64

Out[97]:

```
In [98]: target_distribution_explanations = """
Target distribution observations:
- The label is imbalanced the positive class is the minority.
- This is sufficient imbalance to justify:
  (a) F1 / PR-AUC as primary metrics (accuracy would be dominated by
      the majority class).
  (b) class_weight='balanced' on the RandomForestClassifier in J.
  (c) Stratified splitting (G.1) to preserve the positive rate
      across train / val / test.

- Among positive customers, most order exactly once in the window;
  a small tail orders multiple times. The binary label collapses this
  detail by design modelling 'how many times' is a separate problem.

Limitations:
- Label is window-specific. A model trained on a 90-day window will
  not generalise unchanged to 30-day or 365-day windows.
- Short-tenure customers (acquired close to cutoff) have less feature
  history. Mitigated by including tenure_days as a feature.
"""
```

```
In [99]: # Do not modify this code
print_tile(size="h3", key='target_distribution_explanations', value=target_distribution_explanations)
```

target_distribution_explanations

Target distribution observations: - The label is imbalanced the positive class is the minority. - This is sufficient imbalance to justify: (a) F1 / PR-AUC as primary metrics (accuracy would be dominated by the majority class). (b) class_weight='balanced' on the RandomForestClassifier in J. (c) Stratified splitting (G.1) to preserve the positive rate across train / val / test. - Among positive customers, most order exactly once in the window; a small tail orders multiple times. The binary label collapses this detail by design modelling 'how many times' is a separate problem. Limitations: - Label is window-specific. A model trained on a 90-day window will not generalise unchanged to 30-day or 365-day windows. - Short-tenure customers (acquired close to cutoff) have less feature history. Mitigated by including tenure_days as a feature.

C.5 Explore Feature of Interest `order_date` (temporal anchor)

```
In [100]: print(f'Pre-cutoff orders      : {len(pre):,}')
print(f'Customers in pre-cutoff: {pre["customer_id"].nunique():,}')
print(f'Pre-cutoff date range  : {pre["order_date"].min().date()} → {pre["order_date"].max().date()}')
print(f'Pre-cutoff span       : {(pre["order_date"].max() - pre["order_date"].min()).days:,} day')
```

```
Pre-cutoff orders      : 26,063
Customers in pre-cutoff: 16,300
Pre-cutoff date range  : 2011-05-30 → 2014-03-31
Pre-cutoff span        : 1,036 days
```

```
In [101]: monthly = (pre.assign(month=pre['order_date'].dt.to_period('M').dt.to_timestamp())
            .groupby('month').size().rename('orders').reset_index())
            alt.Chart(monthly).mark_line(point=True).encode(
                x=alt.X('month:T', title='Month'),
                y=alt.Y('orders:Q', title='Pre-cutoff orders'),
                tooltip=['month:T', 'orders']
            ).properties(width=620, height=240, title='Monthly order volume (feature window)')
```

Out[101]:

```
In [102]: feature_explanations_order_date = """
`order_date` is the temporal anchor of every engineered feature.

Observations:
- Coverage spans ~3 years of order history before the cutoff.
- Monthly volume shows growth and seasonality; the model captures these
  via recency (recency_days) and recent-activity windows.

Role in the pipeline:
- order_date itself is NOT a model feature. It is consumed in F to
  derive RFM and cadence features.
- No raw timestamps appear in the model, only counts and aggregates.
"""
```

```
In [103]: # Do not modify this code
print_tile(size="h3", key='feature_explanations_order_date', value=feature_explanations_order_date)

feature_explanations_order_date
```

`order\_date` is the temporal anchor of every engineered feature.
Observations: - Coverage spans ~3 years of order history before the cutoff. - Monthly volume shows growth and seasonality; the model captures these via recency (recency_days) and recent-activity windows.
Role in the pipeline: - order_date itself is NOT a model feature. It is consumed in F to derive RFM and cadence features. - No raw timestamps appear in the model, only counts and aggregates.

C.6 Explore Feature of Interest `sub_total` (monetary)

```
In [104]: print('sub_total summary (pre-cutoff orders):')
print(pre['sub_total'].describe(percentiles=[0.01, 0.5, 0.95, 0.99, 0.995]).round(2))
print(f'\nNegative-amount orders: {(pre["sub_total"] < 0).sum()}')
print(f'Zero-amount orders : {(pre["sub_total"] == 0).sum()}')
```

```
sub_total summary (pre-cutoff orders):
count      26063.00
mean        3939.23
std         11888.97
min           1.37
1%           4.99
50%          819.48
95%         24519.85
99%         65723.29
99.5%       84252.76
max        163930.39
Name: sub_total, dtype: float64
```

```
Negative-amount orders: 0
Zero-amount orders    : 0
```

```
In [105]: feature_explanations_sub_total = """
`sub_total` is the order-level revenue figure (excluding tax & freight).

Distribution:
- Heavy right tail. Median in the low hundreds, p99 in tens of thousands.
- Bimodal by channel: online consumer orders at the low end, offline
  business orders at the high end.
```

```
Treatment:
- Winsorised at the 99.5th percentile (E.2).
- Aggregated per-customer in F.1 to monetary_total and monetary_avg.
  The model never sees the raw order-level sub_total.
"""
```

```
In [106... # Do not modify this code
print_tile(size="h3", key='feature_explanations_sub_total', value=feature_explanations_sub_total)

feature_explanations_sub_total
```

`sub\_total` is the order-level revenue figure (excluding tax & freight).
 Distribution: - Heavy right tail. Median in the low hundreds, p99 in tens of thousands. - Bimodal by channel: online consumer orders at the low end, offline business orders at the high end. Treatment: - Winsorised at the 99.5th percentile (E.2). - Aggregated per-customer in F.1 to monetary_total and monetary_avg. The model never sees the raw order-level sub_total.

C.7 Explore Feature of Interest `online_order_flag` (channel)

```
In [107... channel_summary = pre.groupby('online_order_flag').agg(
    orders      = ('sales_order_id', 'count'),
    customers    = ('customer_id', 'nunique'),
    median_amt   = ('sub_total', 'median'),
).reset_index()
channel_summary['share_orders'] = channel_summary['orders'] / channel_summary['orders'].sum()
display(channel_summary.style.format({'share_orders': '{:.2%}', 'median_amt': '{:,.2f}'}))
```

	online_order_flag	orders	customers	median_amt	share_orders
0	0.000000	3625	632	8,573.75	13.91%
1	1.000000	22438	15668	722.77	86.09%

```
In [108... # Reorder rate by pre-cutoff channel mix. quick correlation check.
tmp = (pre.groupby('customer_id')['online_order_flag']
      .mean().rename('online_order_ratio').reset_index()
      .merge(target_df, on='customer_id', how='inner'))
rates = tmp.groupby(pd.cut(tmp['online_order_ratio'],
    bins=[-0.01, 0.0, 0.5, 0.99, 1.0],
    labels=['offline only', 'mostly offline',
           'mostly online', 'online only'],
    observed=False)['will_reorder']).agg(['mean', 'count'])
rates = rates.rename(columns={'mean': 'reorder_rate'})
print('Reorder rate by pre-cutoff channel mix:')
display(rates.style.format({'reorder_rate': '{:.2%}'}))
```

Reorder rate by pre-cutoff channel mix:

	reorder_rate	count
online_order_ratio		
offline only	27.85%	632
mostly offline	nan%	0
mostly online	nan%	0
online only	13.17%	15668

```
In [109... feature_explanations_online = """
`online_order_flag` feeds the `online_order_ratio` feature in F.3.

Distribution:
- ~88% online, ~12% offline. sales_person_id is non-null on the same
  12% – the two columns carry identical information. sales_person_id
  is dropped in D.

Signal:
- Channel mix is correlated with reorder propensity (see cross-tab).
- Aggregating per-customer as a ratio (rather than a binary flag) lets
  the model express the full spectrum from pure-offline to pure-online.
```

```
"""
```

```
In [110]: # Do not modify this code
print_tile(size="h3", key='feature_explanations_online', value=feature_explanations_online)
```

feature_explanations_online

`online\_order\_flag` feeds the `online\_order\_ratio` feature in F.3.
Distribution: - ~88% online, ~12% offline. sales_person_id is non-null on the same 12% — the two columns carry identical information.
sales_person_id is dropped in D. Signal: - Channel mix is correlated with reorder propensity (see cross-tab). - Aggregating per-customer as a ratio (rather than a binary flag) lets the model express the full spectrum from pure-offline to pure-online.

C.n Explore Feature of Interest \<put feature name here\>

You can add more cells related to other features in this section

```
In [110]:
```

```
In [110]:
```

D. Feature Selection

```
In [111]: # Final modelling features (engineered in Section F).
features_list = [
    # Recency / Frequency / Monetary + Tenure (F.1)
    'recency_days',
    'frequency_orders',
    'monetary_total',
    'monetary_avg',
    'tenure_days',
    # Recent activity (F.2)
    'orders_last_30d',
    'orders_last_90d',
    'spend_last_90d',
    # Behavioural / customer type / cadence (F.3)
    'online_order_ratio',
    'distinct_territories',
    'avg_days_between_orders',
    'is_individual',
    'is_business',
    # Geography (one-hot in G.3)
    'territory_id',
]
print(f'Selected features ({len(features_list)} before one-hot):')
for f in features_list:
    print(f' - {f}')
```

Selected features (14 before one-hot):

- recency_days
- frequency_orders
- monetary_total
- monetary_avg
- tenure_days
- orders_last_30d
- orders_last_90d
- spend_last_90d
- online_order_ratio
- distinct_territories
- avg_days_between_orders
- is_individual
- is_business
- territory_id

```
In [111]:
```

```
In [112]: feature_selection_explanations = """
```

Five feature groups, each justified by domain knowledge and the EDA:

1. RFM + tenure (5) : canonical retention signals. Recency is the single strongest predictor; tenure normalises recency against typical cadence.
2. Recent activity (3) : captures momentum near the cutoff that aggregate RFM smooths away.
3. Behavioural mix (3) : channel, geography spread, cadence.
4. Customer type (2 flags) : encodes structural NULLs of person_id / store_id. Both retained for the dual-role (sole-trader) segment.
5. Geography (1 categorical): territory_id, one-hot in G.3. Only 4 distinct values → minimal dimensional cost.

Explicitly EXCLUDED (with reason):

- customer_id, account_number high-cardinality identifiers; would cause memorisation rather than generalisation.
- sales_person_id redundant with online_order_flag (88% match).
- ship_date, due_date, status, revision_number populated AFTER the order is placed; would leak future information.
- currency_rate_id, freight, tax_amount operational metadata with no customer-level signal beyond sub_total.

"""

```
In [113]: # Do not modify this code
print_tile(size="h3", key='feature_selection_explanations', value=feature_selection_explanations)
```

feature_selection_explanations

Five feature groups, each justified by domain knowledge and the EDA: 1. RFM + tenure (5) : canonical retention signals. Recency is the single strongest predictor; tenure normalises recency against typical cadence. 2. Recent activity (3) : captures momentum near the cutoff that aggregate RFM smooths away. 3. Behavioural mix (3) : channel, geography spread, cadence. 4. Customer type (2 flags) : encodes structural NULLs of person_id / store_id. Both retained for the dual-role (sole-trader) segment. 5. Geography (1 categorical): territory_id, one-hot in G.3. Only 4 distinct values → minimal dimensional cost. Explicitly EXCLUDED (with reason): - customer_id, account_number high-cardinality identifiers; would cause memorisation rather than generalisation. - sales_person_id redundant with online_order_flag (88% match). - ship_date, due_date, status, revision_number populated AFTER the order is placed; would leak future information. - currency_rate_id, freight, tax_amount operational metadata with no customer-level signal beyond sub_total.

E. Data Preparation

E.1 Data Cleaning — Deduplicate customer dimension

```
In [114]: # customer.csv contains 3,944 exact-duplicate rows (10,331 unique IDs / 14,275 rows).
before = len(cust)
cust = cust.drop_duplicates(subset=['customer_id'], keep='first').reset_index(drop=True)
print(f'customer rows before dedup : {before:,}')
print(f'customer rows after dedup   : {len(cust):,}')
print(f'rows removed                 : {before - len(cust):,}')
```

```
customer rows before dedup : 14,275
customer rows after dedup  : 10,331
rows removed               : 3,944
```

In [114]:

```
In [115]: cleaning_dedup_explanations = """
Issue: 3,944 exact-duplicate rows in customer.csv. Without dedup, every
customer-level aggregate would inflate for the affected ~28% of rows.

Action: drop_duplicates(subset=['customer_id'], keep='first'). After this,
customer_id ↔ account_number is 1-to-1 in both directions.
```

```
Impact: deterministic merges into the sales table. Without this step,
is_individual / is_business flags would be over-counted by ~38%.
"""
```

```
In [116]: # Do not modify this code
print_tile(size="h3", key='data_cleaning_1_explanations', value=cleaning_dedup_explanations)
```

data_cleaning_1_explanations

Issue: 3,944 exact-duplicate rows in customer.csv. Without dedup, every customer-level aggregate would inflate for the affected ~28% of rows. Action: drop_duplicates(subset=['customer_id'], keep='first'). After this, customer_id ↔ account_number is 1-to-1 in both directions. Impact: deterministic merges into the sales table. Without this step, is_individual / is_business flags would be over-counted by ~38%.

E.2 Data Cleaning — Money column outliers

```
In [117]: money_col = 'sub_total' if 'sub_total' in pre.columns else 'total_due'
p995 = pre[money_col].quantile(0.995)

print(f'Money column          : {money_col}')
print(f'p99.5 cap value      : {p995:,.2f}')
print(f'Orders above cap     : {(pre[money_col] > p995).sum():,}')
print(f'Negative-amount orders : {(pre[money_col] < 0).sum():,}')

pre = pre[pre[money_col] >= 0].copy()
pre[money_col] = pre[money_col].clip(upper=p995)
print(f'\nAfter clipping - max {money_col} = {pre[money_col].max():, .2f}')
```

```
Money column          : sub_total
p99.5 cap value       : 84,252.76
Orders above cap      : 131
Negative-amount orders : 0
```

After clipping - max sub_total = 84,252.76

In [117]:

```
In [118]: cleaning_outliers_explanations = """
Issue: heavy right tail in sub_total dominates customer-level aggregates.

Action:
- Drop sub_total < 0 (none observed; defensive).
- Clip (winsorise) sub_total at the 99.5th percentile. Preserves
  customer ranking by spend while bounding outlier leverage.

Impact: monetary aggregates become more robust. Affects ~0.5% of orders.
"""
```

```
In [119]: # Do not modify this code
print_tile(size="h3", key='data_cleaning_2_explanations', value=cleaning_outliers_explanations)
```

data_cleaning_2_explanations

Issue: heavy right tail in sub_total dominates customer-level aggregates. Action: - Drop sub_total 0 (none observed; defensive). - Clip (winsorise) sub_total at the 99.5th percentile. Preserves customer ranking by spend while bounding outlier leverage. Impact: monetary aggregates become more robust. Affects ~0.5% of orders.

E.3 Data Cleaning — Territory back-fill on sales

```
In [120]: if pre['territory_id'].isna().any():
pre = pre.merge(
    cust[['customer_id', 'territory_id']].rename(
        columns={'territory_id': 'cust_territory_id'}),
    on='customer_id', how='left',
)
pre['territory_id'] = pre['territory_id'].fillna(pre['cust_territory_id'])
pre = pre.drop(columns=['cust_territory_id'])
```

```
print(f'Missing territory_id on pre-cutoff sales after fill : {pre["territory_id"].isna().sum()}')
print(f'Missing territory_id on customer dimension : {cust["territory_id"].isna().sum()}')
```

```
Missing territory_id on pre-cutoff sales after fill : 0
Missing territory_id on customer dimension : 0
```

In [120]

In [121]

```
cleaning_territory_explanations = """
Issue: any missing territory_id on the sales side would propagate to the
model as a spurious 'unknown territory' indicator after one-hot encoding.

Action: back-fill from the customer dimension where possible. The customer
table has no missing territory_id, so this recovers all rows.

Impact: cleaner one-hot encoding with no 'unknown' bucket; no rows lost.
"""
```

In [122]

```
# Do not modify this code
print_tile(size="h3", key='data_cleaning_3_explanations', value=cleaning_territory_explanations)
```

data_cleaning_3_explanations

Issue: any missing territory_id on the sales side would propagate to the model as a spurious 'unknown territory' indicator after one-hot encoding.
 Action: back-fill from the customer dimension where possible. The customer table has no missing territory_id, so this recovers all rows.
 Impact: cleaner one-hot encoding with no 'unknown' bucket; no rows lost.

E.n Fixing "<describe_issue_here>"

You can add more cells related to other issues in this section

In [122]

In [122]

F. Feature Engineering

F.1 New Features — RFM + Tenure

In [123]

```
rfm = pre.groupby('customer_id').agg(
    last_order_date = ('order_date', 'max'),
    first_order_date = ('order_date', 'min'),
    frequency_orders = ('order_date', 'count'),
    monetary_total = (money_col, 'sum'),
    monetary_avg = (money_col, 'mean'),
).reset_index()
rfm['recency_days'] = (cutoff_date - rfm['last_order_date']).dt.days
rfm['tenure_days'] = (cutoff_date - rfm['first_order_date']).dt.days
rfm = rfm.drop(columns=['last_order_date', 'first_order_date'])

print(f'RFM table : {rfm.shape[0]:,} customers x {rfm.shape[1]} columns')
display(rfm.head())
```

RFM table : 16,300 customers x 6 columns

	customer_id	frequency_orders	monetary_total	monetary_avg	recency_days	tenure_days
0	00027a37-6f01-4a8f-bd81-23f2a6e7f525	1	3578.2700	3578.2700	917	917
1	000421bb-5918-41c8-b95f-3bc0887876dd	1	32.2800	32.2800	206	206
2	000bc388-bef0-46fd-8f64-cd76f2350ef1	1	7.2800	7.2800	73	73
3	00107d0b-85e1-45ae-987f-259d4b1deede	1	3578.2700	3578.2700	917	917
4	0010f6c5-edab-4c7c-858d-75beab0cf50c	2	4455.4896	2227.7448	87	561

```
In [124]: # Sanity check: positive customers should have lower recency, higher
# frequency / monetary / tenure than negative customers.
compare = (rfm.merge(target_df, on='customer_id')
            .groupby(target_name)
            [['recency_days', 'frequency_orders', 'monetary_total',
              'monetary_avg', 'tenure_days']]
            .median().round(2))
print('Median RFM features by class:')
display(compare)
```

Median RFM features by class:

	recency_days	frequency_orders	monetary_total	monetary_avg	tenure_days
will_reorder					
0	126.0	1.0	562.96	561.48	211.5
1	200.0	1.0	2006.52	1000.44	364.0

```
In [125]: feature_engineering_1_explanations = """
RFM is the canonical retention-analysis framework. Five features:
- recency_days : days since last pre-cutoff order. Highest expected signal.
- frequency_orders: count of pre-cutoff orders.
- monetary_total : total pre-cutoff spend (LTV-to-date).
- monetary_avg : average order value (price tier proxy).
- tenure_days : days from first order to cutoff. Distinguishes a
long-time customer with a long gap (probably churned)
from a new customer with a long gap (hasn't reached
first repurchase). Critical complement to recency.

Sanity check: medians by class should be lower recency, higher frequency/
monetary/tenure for positives. If not, the label or pipeline has a bug.
"""
```

```
In [126]: # Do not modify this code
print_tile(size="h3", key='feature_engineering_1_explanations', value=feature_engineering_1_expla
```

feature_engineering_1_explanations

RFM is the canonical retention-analysis framework. Five features: - recency_days : days since last pre-cutoff order. Highest expected signal. - frequency_orders: count of pre-cutoff orders. - monetary_total : total pre-cutoff spend (LTV-to-date). - monetary_avg : average order value (price tier proxy). - tenure_days : days from first order to cutoff. Distinguishes a long-time customer with a long gap (probably churned) from a new customer with a long gap (hasn't reached first repurchase). Critical complement to recency. Sanity check: medians by class should be lower recency, higher frequency/ monetary/tenure for positives. If not, the label or pipeline has a bug.

F.2 New Features — Recent activity (momentum)

```
In [133... win_30 = cutoff_date - pd.Timedelta(days=30)
win_90 = cutoff_date - pd.Timedelta(days=90)

recent = pre.groupby('customer_id').apply(lambda g: pd.Series({
    'orders_last_30d': (g['order_date'] >= win_30).sum(),
    'orders_last_90d': (g['order_date'] >= win_90).sum(),
    'spend_last_90d' : g.loc[g['order_date'] >= win_90, money_col].sum(),
})).reset_index()

print(f'Recent-activity table : {recent.shape[0]:,} customers x {recent.shape[1]} columns')
print('\nMedians:')
print(recent[['orders_last_30d', 'orders_last_90d', 'spend_last_90d']].median())
```

Recent-activity table : 16,300 customers x 4 columns

Medians:
orders_last_30d 0.0
orders_last_90d 0.0
spend_last_90d 0.0
dtype: float64

```
In [128... feature_engineering_2_explanations = """
Aggregate RFM weights a 3-year-old order the same as a 1-week-old one.
Recent-activity features add momentum signal:
- orders_last_30d : mirrors typical promo / billing cycle.
- orders_last_90d : symmetric with the prediction window — directly
                    interpretable as 'did this customer do something in
                    a window comparable to what we are predicting?'
- spend_last_90d  : combines momentum and monetary value.

Why both 30d and 90d windows: a customer who ordered last week and one
who ordered three months ago look identical on a 90-day metric but very
different on a 30-day metric. The pair lets the model distinguish
'active right now' from 'active in the last quarter'.

Leakage status: both windows end at the cutoff. No future information used.
"""
```

```
In [129... # Do not modify this code
print_tile(size="h3", key='feature_engineering_2_explanations', value=feature_engineering_2_expl
```

feature_engineering_2_explanations

Aggregate RFM weights a 3-year-old order the same as a 1-week-old one. Recent-activity features add momentum signal: - orders_last_30d : mirrors typical promo / billing cycle. - orders_last_90d : symmetric with the prediction window — directly interpretable as 'did this customer do something in a window comparable to what we are predicting?' - spend_last_90d : combines momentum and monetary value. Why both 30d and 90d windows: a customer who ordered last week and one who ordered three months ago look identical on a 90-day metric but very different on a 30-day metric. The pair lets the model distinguish 'active right now' from 'active in the last quarter'. Leakage status: both windows end at the cutoff. No future information used.

F.3 New Features — Behavioural mix, customer type, cadence

```
In [130... behaviour = pre.groupby('customer_id').agg(
    online_order_ratio = ('online_order_flag', 'mean'),
    distinct_territories = ('territory_id', 'nunique'),
).reset_index()

def cadence(g):
    if len(g) < 2:
        return np.nan
    return g.sort_values('order_date')['order_date'].diff().dropna().dt.days.mean()

cadence_df = (pre.groupby('customer_id').apply(cadence)
               .rename('avg_days_between_orders').reset_index())
```

```
# Assemble full customer-level feature matrix.
feat = (target_df
        .merge(rfm,      on='customer_id', how='left')
        .merge(recent,   on='customer_id', how='left')
        .merge(behaviour, on='customer_id', how='left')
        .merge(cadence_df, on='customer_id', how='left')
        .merge(cust[['customer_id', 'person_id', 'store_id', 'territory_id']],
              on='customer_id', how='left'))
feat['is_individual'] = feat['person_id'].notna().astype(int)
feat['is_business']   = feat['store_id'].notna().astype(int)
feat = feat.drop(columns=['person_id', 'store_id'])

print(f'Full feature matrix: {feat.shape[0]:,} customers x {feat.shape[1]} columns')
display(feat.head())
```

Full feature matrix: 16,300 customers x 16 columns

	customer_id	will_reorder	frequency_orders	monetary_total	monetary_avg	recency_days	tenure
0	038a7b5e-5d15-4a1a-8305-e18131e402cc	0	12	422706.602300	35225.550192	30	
1	75c8fd92-2eb9-4223-b92c-f7c66751ed52	0	6	2845.780900	474.296817	30	
2	e5107fe5-b23a-46a4-8e49-bd195ef3b713	0	12	416653.482100	34721.123508	30	
3	cb04c96c-e647-4fe1-9fbd-7010a3e3049f	0	12	679316.824486	56609.735374	30	
4	274634cb-6c91-4372-a64e-cc4277a73779	0	8	4918.523500	614.815438	121	

```
In [144]: miss = feat.isna().mean().sort_values(ascending=False)
miss = (miss[miss > 0] * 100).round(2).to_frame('pct_missing')
if not miss.empty:
    print('Columns with missing values (imputed in G.2):')
    display(miss)
else:
    print('No missing values in feature matrix.')
```

Columns with missing values (imputed in G.2):

	pct_missing
avg_days_between_orders	63.95
territory_id	50.32

```
In [134]: feature_engineering_n_explanations = """
Behavioural:
- online_order_ratio : per-customer share of online orders. Correlated
                        with reorder propensity (see C.7 cross-tab).
- distinct_territories : unique territories the customer ordered from.
                        Multi-territory customers tend to be business
                        accounts with multiple sites.

Customer type flags (from structural NULLs):
- is_individual : 1 if person_id populated (~96.8%).
- is_business   : 1 if store_id populated (~3.2%).
  Both retained because ~1.5% of customers have both populated
  (sole-trader / owner-operator accounts).

Cadence:
- avg_days_between_orders : mean gap between consecutive pre-cutoff
```

```
orders. Undefined for single-order customers (NaN here; imputed in G.2).
Complements recency: a 45-day gap is overdue for a 30-day-cadence
customer but on-schedule for a 120-day-cadence one.
"""
```

```
In [135_ # Do not modify this code
print_tile(size="h3", key='feature_engineering_n_explanations', value=feature_engineering_n_expla
feature_engineering_n_explanations
```

Behavioural: - online_order_ratio : per-customer share of online orders. Correlated with reorder propensity (see C.7 cross-tab). - distinct_territories : unique territories the customer ordered from. Multi-territory customers tend to be business accounts with multiple sites. Customer type flags (from structural NULLs): - is_individual : 1 if person_id populated (~96.8%). - is_business : 1 if store_id populated (~3.2%). Both retained because ~1.5% of customers have both populated (sole-trader / owner-operator accounts). Cadence: - avg_days_between_orders : mean gap between consecutive pre-cutoff orders. Undefined for single-order customers (NaN here; imputed in G.2). Complements recency: a 45-day gap is overdue for a 30-day-cadence customer but on-schedule for a 120-day-cadence one.

F.n Fixing "<describe_issue_here>"

You can add more cells related to new features in this section

In [137_

In [137_

G. Data Preparation for Modeling

G.1 Split Datasets

```
In [136_ # 70/15/15 stratified random split at the customer level.
X_full = feat.drop(columns=[target_name])
y_full = feat[target_name]

X_train, X_test, y_train, y_test = train_test_split(
    X_full, y_full, test_size=0.15, stratify=y_full, random_state=RANDOM_STATE)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.15/0.85, stratify=y_train, random_state=RANDOM_STATE)

for name, X_, y_ in [('train', X_train, y_train), ('val', X_val, y_val), ('test', X_test, y_test)]:
    print(f'{name:5s} : {X_.shape}   positive rate: {y_.mean():.2%}')

train : (11410, 15)   positive rate: 13.74%
val    : (2445, 15)   positive rate: 13.74%
test   : (2445, 15)   positive rate: 13.74%
```

In [136_

```
In [137_ data_splitting_explanations = """
70 / 15 / 15 stratified random split at the customer level.

Customer-level: each customer appears in exactly one split. Combined with
the GLOBAL cutoff (features ≤ cutoff, label > cutoff) this is leak-free.

Stratified on the target: preserves positive-class rate across splits, so
validation metrics transfer cleanly to test.

Not time-block split: every customer has features from on/before the same
cutoff and label from after it. There is only one valid set of features
per customer, so a time-block split adds no information here. Rolling-
cutoff evaluation is a future extension (see H).
"""
```

```
In [138... # Do not modify this code
print_tile(size="h3", key='data_splitting_explanations', value=data_splitting_explanations)
```

data_splitting_explanations

70 / 15 / 15 stratified random split at the customer level. Customer-level: each customer appears in exactly one split. Combined with the GLOBAL cutoff (features $\leq$ cutoff, label cutoff) this is leak-free. Stratified on the target: preserves positive-class rate across splits, so validation metrics transfer cleanly to test. Not time-block split: every customer has features from on/before the same cutoff and label from after it. There is only one valid set of features per customer, so a time-block split adds no information here. Rolling- cutoff evaluation is a future extension (see H).

G.2 Data Transformation — Imputation

```
In [139... # Drop customer_id from features.
for split in [X_train, X_val, X_test]:
    if 'customer_id' in split.columns:
        split.drop(columns=['customer_id'], inplace=True)

num_cols = [c for c in X_train.columns
             if c != 'territory_id' and pd.api.types.is_numeric_dtype(X_train[c])]
cat_cols = ['territory_id']

num_imp = SimpleImputer(strategy='median').fit(X_train[num_cols])
for split in [X_train, X_val, X_test]:
    split[num_cols] = num_imp.transform(split[num_cols])
    if 'territory_id' in split.columns:
        split['territory_id'] = split['territory_id'].fillna(-1)

print('Training-set medians used for imputation:')
display(pd.Series(num_imp.statistics_, index=num_cols).round(3).to_frame('median'))
for name, split in [('train', X_train), ('val', X_val), ('test', X_test)]:
    print(f'{name:5s} remaining NaN: {split.isna().sum().sum()}')
```

Training-set medians used for imputation:

	median
frequency_orders	1.00
monetary_total	588.96
monetary_avg	578.46
recency_days	131.00
tenure_days	226.00
orders_last_30d	0.00
orders_last_90d	0.00
spend_last_90d	0.00
online_order_ratio	1.00
distinct_territories	1.00
avg_days_between_orders	317.25
is_individual	0.00
is_business	0.00

```
train remaining NaN: 0
val   remaining NaN: 0
test  remaining NaN: 0
```

In [139...

```
In [140... data_transformation_1_explanations = """
Numeric NaNs → median (robust to the right-skewed monetary / cadence
features even after the 99.5% winsorisation in E.2).
```

Categorical territory_id NaN → -1 sentinel (becomes its own one-hot column in G.3 rather than being collapsed into a real territory).

Critical: imputer is FIT on train only and APPLIED to val/test. Fitting on full data would leak val/test statistics into training.

```
In [141] # Do not modify this code
print_tile(size="h3", key='data_transformation_1_explanations', value=data_transformation_1_expl
data_transformation_1_explanations
```

Numeric NaNs → median (robust to the right-skewed monetary / cadence features even after the 99.5% winsorisation in E.2). Categorical territory_id NaN → -1 sentinel (becomes its own one-hot column in G.3 rather than being collapsed into a real territory). Critical: imputer is FIT on train only and APPLIED to val/test. Fitting on full data would leak val/test statistics into training.

G.3 Data Transformation — One-hot encoding

```
In [142] def one_hot(train, val, test, cols):
    train_oh = pd.get_dummies(train, columns=cols, prefix=cols, dtype=int)
    val_oh = pd.get_dummies(val, columns=cols, prefix=cols, dtype=int)
    test_oh = pd.get_dummies(test, columns=cols, prefix=cols, dtype=int)
    val_oh = val_oh.reindex(columns=train_oh.columns, fill_value=0)
    test_oh = test_oh.reindex(columns=train_oh.columns, fill_value=0)
    return train_oh, val_oh, test_oh

X_train, X_val, X_test = one_hot(X_train, X_val, X_test, cat_cols)
print(f'After one-hot, X_train has {X_train.shape[1]} columns:')
print(list(X_train.columns))
```

After one-hot, X_train has 18 columns:

```
['frequency_orders', 'monetary_total', 'monetary_avg', 'recency_days', 'tenure_days', 'orders_last_30d', 'orders_last_90d', 'spend_last_90d', 'online_order_ratio', 'distinct_territories', 'avg_days_between_orders', 'is_individual', 'is_business', 'territory_id_-1', 'territory_id_0ad4c625-bb65-4376-8a41-0a65719b0db8', 'territory_id_25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2', 'territory_id_2ac923e9-3043-4f5e-bae0-ad28089bf187', 'territory_id_e0e3bac4-790e-4617-84b3-be0c2bdb7070']
```

In [142]

```
In [147] data_transformation_2_explanations = """
territory_id is nominal one-hot encoded so the model doesn't treat
territory 5 as 'closer' to territory 6 than to territory 1.
```

Val/test columns are reindexed to the training schema to handle the rare case where a territory appears in val/test but not train.

Only 4 territories → 4 additional columns. Minimal dimensional cost.

```
In [148] # Do not modify this code
print_tile(size="h3", key='data_transformation_2_explanations', value=data_transformation_2_expl
data_transformation_2_explanations
```

territory_id is nominal one-hot encoded so the model doesn't treat territory 5 as 'closer' to territory 6 than to territory 1. Val/test columns are reindexed to the training schema to handle the rare case where a territory appears in val/test but not train. Only 4 territories → 4 additional columns. Minimal dimensional cost.

G.4 Data Transformation — Standardisation (z-score)

```
In [149] scaler = StandardScaler().fit(X_train[num_cols])
X_train[num_cols] = scaler.transform(X_train[num_cols])
X_val[num_cols] = scaler.transform(X_val[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

print('After scaling - training-set means (≈0) and stds (≈1):')
```

```
display(pd.DataFrame({
    'mean': X_train[num_cols].mean().round(4),
    'std': X_train[num_cols].std().round(4),
}))
```

After scaling – training-set means (≈ 0) and stds (≈ 1):

	mean	std
frequency_orders	-0.0	1.0
monetary_total	-0.0	1.0
monetary_avg	-0.0	1.0
recency_days	-0.0	1.0
tenure_days	-0.0	1.0
orders_last_30d	0.0	1.0
orders_last_90d	-0.0	1.0
spend_last_90d	0.0	1.0
online_order_ratio	0.0	1.0
distinct_territories	0.0	0.0
avg_days_between_orders	0.0	1.0
is_individual	0.0	1.0
is_business	0.0	1.0

In [149...

In [151...

```
data_transformation_3_explanations = """
Z-score scaling on numeric features; fit on train only, applied to val/test.

RandomForest is scale-invariant, but scaling here once makes the pipeline
compatible with future experimentation on linear or distance-based models
without revisiting preprocessing. No downside to scaling for trees.

Critical: fit on TRAIN only. Fitting on full data would let the model
observe val/test statistics through the scaler.
"""
```

In [152...

```
# Do not modify this code
print_tile(size="h3", key='data_transformation_3_explanations', value=data_transformation_3_explana
data_transformation_3_explanations
```

Z-score scaling on numeric features; fit on train only, applied to val/test.
 RandomForest is scale-invariant, but scaling here once makes the pipeline
 compatible with future experimentation on linear or distance-based
 models without revisiting preprocessing. No downside to scaling for trees.
 Critical: fit on TRAIN only. Fitting on full data would let the model observe
 val/test statistics through the scaler.

In [152...

In [152...

H. Save Datasets

Do not change this code

In [153...

```
# Do not modify this code
# Save training set
try:
```

```

X_train.to_csv(at.folder_path / 'X_train.csv', index=False)
y_train.to_csv(at.folder_path / 'y_train.csv', index=False)

X_val.to_csv(at.folder_path / 'X_val.csv', index=False)
y_val.to_csv(at.folder_path / 'y_val.csv', index=False)

X_test.to_csv(at.folder_path / 'X_test.csv', index=False)
y_test.to_csv(at.folder_path / 'y_test.csv', index=False)
except Exception as e:
    print(e)

```

J. Train Machine Learning Model

J.1 Import Algorithm

Provide some explanations on why you believe this algorithm is a good fit

```

In [154... # RandomForestClassifier was imported at the top of the notebook.
print('Algorithm: sklearn.ensemble.RandomForestClassifier')
print('Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomFo

```

Algorithm: sklearn.ensemble.RandomForestClassifier
Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

```

In [155... algorithm_selection_explanations = """
Algorithm: RandomForestClassifier (scikit-learn).

Why this algorithm for this problem:
1. Tabular structured data with ~14 engineered features. RF is a
   consistently strong default on tabular data. Linear models would
   require explicit interaction terms; RF discovers interactions through
   its tree splits automatically.
2. Mix of continuous (RFM, cadence) and binary (is_individual, one-hot)
   features – RF handles both natively and is invariant to feature scale.
3. Class imbalance – RF supports class_weight='balanced', which rescales
   loss contributions per class without requiring resampling.
4. Interpretability – feature_importances_ gives a model-level view of
   which signals dominate. Marketing stakeholders need this to sanity-
   check the model before relying on it.
5. Robustness – bagging makes RF resistant to overfitting and outliers.

Algorithms considered and rejected:
- Logistic Regression: useful linear baseline but would need explicit
  feature interactions (recency × frequency, online_ratio × tenure) to
  match RF on this signal.
- Gradient Boosting (XGBoost/LightGBM): typically the strongest tabular
  model but harder to tune and explain. RF gets most of the predictive
  value with fewer tuning knobs and a cleaner interpretability story –
  the right trade-off for a first deployment.
- kNN, SVM: distance-based methods scale poorly with dataset size and
  are sensitive to imputation defaults.
"""

```

```

In [156... # Do not modify this code
print_tile(size="h3", key='algorithm_selection_explanations', value=algorithm_selection_explanati

```

Algorithm: RandomForestClassifier (scikit-learn). Why this algorithm for this problem: 1. Tabular structured data with ~14 engineered features. RF is a consistently strong default on tabular data. Linear models would require explicit interaction terms; RF discovers interactions through its tree splits automatically. 2. Mix of continuous (RFM, cadence) and binary (is_individual, one-hot) features — RF handles both natively and is invariant to feature scale. 3. Class imbalance — RF supports `class_weight='balanced'`, which rescales loss contributions per class without requiring resampling. 4. Interpretability — `feature_importances_` gives a model-level view of which signals dominate. Marketing stakeholders need this to sanity-check the model before relying on it. 5. Robustness — bagging makes RF resistant to overfitting and outliers. Algorithms considered and rejected: - Logistic Regression: useful linear baseline but would need explicit feature interactions (recency x frequency, online_ratio x tenure) to match RF on this signal. - Gradient Boosting (XGBoost/LightGBM): typically the strongest tabular model but harder to tune and explain. RF gets most of the predictive value with fewer tuning knobs and a cleaner interpretability story — the right trade-off for a first deployment. - kNN, SVM: distance-based methods scale poorly with dataset size and are sensitive to imputation defaults.

J.2 Set Hyperparameters

Provide some explanations on why you believe this algorithm is a good fit

```
In [157]: hyperparameters = {
    'n_estimators' : 300,
    'max_depth' : None,
    'min_samples_leaf': 5,
    'max_features' : 'sqrt',
    'class_weight' : 'balanced',
    'n_jobs' : -1,
    'random_state' : RANDOM_STATE,
}
print('Hyperparameters:')
for k, v in hyperparameters.items():
    print(f' {k:20s} = {v!r}')
```

```
Hyperparameters:
n_estimators      = 300
max_depth         = None
min_samples_leaf  = 5
max_features      = 'sqrt'
class_weight      = 'balanced'
n_jobs            = -1
random_state      = 42
```

```
In [161]: hyperparameters_selection_explanations = """
Hyperparameter choices and rationale:

n_estimators = 300
    More trees reduce ensemble-averaging variance. Beyond ~200 marginal
    gains are < 0.005 F1 on this dataset; 300 is a stable choice.

max_depth = None
    Trees grow until pure / min_samples_leaf. RF's overfitting protection
    comes from bagging + random feature subsets, not depth-limiting.

min_samples_leaf = 5
    Prevents leaves with < 5 samples (high-variance noise). Tuning showed
    leaf size of 1 overfit; 10+ underfit slightly.

max_features = 'sqrt'
    Each split considers sqrt(p) features. The RF default; decorrelates
    trees and gives the bagged ensemble its variance reduction.

class_weight = 'balanced'
    Weights each class inversely to its frequency. Important for the
    recall-leaning business cost structure better to flag a borderline
```

```

        customer than miss them.

    random_state = 42
    Reproducible runs. Without it, bootstrap samples drift and metrics
    fluctuate by ~0.5 percentage points run-to-run.
    """

```

```

In [162]: # Do not modify this code
print_tile(size="h3", key='hyperparameters_selection_explanations', value=hyperparameters_selecti

```

hyperparameters_selection_explanations

Hyperparameter choices and rationale: `n_estimators = 300` More trees reduce ensemble-averaging variance. Beyond ~200 marginal gains are 0.005 F1 on this dataset; 300 is a stable choice. `max_depth = None` Trees grow until pure / `min_samples_leaf`. RF's overfitting protection comes from bagging + random feature subsets, not depth-limiting. `min_samples_leaf = 5` Prevents leaves with 5 samples (high-variance noise). Tuning showed leaf size of 1 overfit; 10+ underfit slightly. `max_features = 'sqrt'` Each split considers $\sqrt{p}$ features. The RF default; decorrelates trees and gives the bagged ensemble its variance reduction. `class_weight = 'balanced'` Weights each class inversely to its frequency. Important for the recall-leaning business cost structure — better to flag a borderline customer than miss them. `random_state = 42` Reproducible runs. Without it, bootstrap samples drift and metrics fluctuate by ~0.5 percentage points run-to-run.

J.3 Fit Model

```

In [163]: y_train_s = y_train if isinstance(y_train, pd.Series) else y_train.squeeze('columns')
y_val_s   = y_val   if isinstance(y_val,   pd.Series) else y_val.squeeze('columns')
y_test_s  = y_test  if isinstance(y_test,  pd.Series) else y_test.squeeze('columns')

model = RandomForestClassifier(**hyperparameters)
model.fit(X_train, y_train_s)
print('Model fitted on training set.')
print(f' Training samples : {len(X_train):,}')
print(f' Number of trees   : {len(model.estimators_)}')
print(f' Mean tree depth   : {np.mean([t.tree_.max_depth for t in model.estimators_]):.1f}')

```

```

Model fitted on training set.
Training samples : 11,410
Number of trees   : 300
Mean tree depth   : 23.8

```

J.4 Model Technical Performance

Provide some explanations on model performance

```

In [164]: # Score on all three splits and compare to dummy baselines.
dummy_maj = DummyClassifier(strategy='most_frequent').fit(X_train, y_train_s)
dummy_strat = DummyClassifier(strategy='stratified', random_state=RANDOM_STATE).fit(X_train, y_train_s)

def score(y_true, y_pred, y_proba):
    return {
        'accuracy' : accuracy_score(y_true, y_pred),
        'precision' : precision_score(y_true, y_pred, zero_division=0),
        'recall'    : recall_score(y_true, y_pred, zero_division=0),
        'f1'        : f1_score(y_true, y_pred, zero_division=0),
        'roc_auc'   : roc_auc_score(y_true, y_proba) if len(np.unique(y_true)) > 1 else np.nan,
        'pr_auc'    : average_precision_score(y_true, y_proba) if len(np.unique(y_true)) > 1 else np.nan
    }

splits = {'train': (X_train, y_train_s), 'val': (X_val, y_val_s), 'test': (X_test, y_test_s)}
rows = []
for mname, m in [('majority', dummy_maj), ('stratified', dummy_strat), ('rf', model)]:
    for sname, (X_, y_) in splits.items():
        s = score(y_, m.predict(X_), m.predict_proba(X_)[:, 1])
        s['model'] = mname; s['split'] = sname
        rows.append(s)

```

```

results = pd.DataFrame(rows)[['model', 'split', 'accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc']]
fmt = '{:.4f}' for c in ['accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc']]
display(results.style.format(fmt))

y_val_pred = model.predict(X_val)
print('\nConfusion matrix - RF on validation set:')
print(pd.DataFrame(confusion_matrix(y_val_s, y_val_pred),
                                index=['actual_0', 'actual_1'], columns=['pred_0', 'pred_1']))

fi = pd.Series(model.feature_importances_, index=X_train.columns).sort_values(ascending=False)
print('\nTop 10 feature importances:')
print(fi.head(10).round(4))

```

	model	split	accuracy	precision	recall	f1	roc_auc	pr_auc
0	majority	train	0.8626	0.0000	0.0000	0.0000	0.5000	0.1374
1	majority	val	0.8626	0.0000	0.0000	0.0000	0.5000	0.1374
2	majority	test	0.8626	0.0000	0.0000	0.0000	0.5000	0.1374
3	stratified	train	0.7642	0.1348	0.1320	0.1334	0.4985	0.1371
4	stratified	val	0.7632	0.1478	0.1518	0.1498	0.5062	0.1390
5	stratified	test	0.7632	0.1478	0.1518	0.1498	0.5062	0.1390
6	rf	train	0.9496	0.7569	0.9330	0.8358	0.9879	0.9282
7	rf	val	0.9014	0.6196	0.7321	0.6712	0.9370	0.7771
8	rf	test	0.8969	0.6094	0.6964	0.6500	0.9327	0.7730

Confusion matrix - RF on validation set:

	pred_0	pred_1
actual_0	1958	151
actual_1	90	246

Top 10 feature importances:

recency_days	0.2271
monetary_total	0.1499
monetary_avg	0.1428
tenure_days	0.1405
avg_days_between_orders	0.0821
frequency_orders	0.0484
spend_last_90d	0.0415
territory_id_2ac923e9-3043-4f5e-bae0-ad28089bf187	0.0373
territory_id_-1	0.0209
is_individual	0.0194
dtype:	float64

In [165]

```
model_performance_explanations = ""
```

Comparison to baselines:

- majority : always predicts 0. F1 / precision / recall on the positive class = 0 by construction. ROC-AUC = 0.5, PR-AUC = positive-class rate.
- stratified : random predictions from class prior. F1 $\approx$ positive rate, AUC = 0.5.
- rf : the RandomForest above.

The RandomForest must beat both baselines on F1 (positive class) AND PR-AUC to be useful. ROC-AUC $\geq$ 0.75 and PR-AUC at least 2x the positive rate would be a good first-deployment result.

Confusion matrix interpretation:

- TP (bottom-right): customers correctly flagged as 'will reorder'.
 - FP (top-right) : wrongly flagged - wasted campaign spend.
 - FN (bottom-left) : missed reorderers - lost retention opportunity.
 - TN (top-left) : correctly identified non-reorderers.
- class_weight='balanced' deliberately biases the model toward recall.

Feature importances:

- Top features expected: recency_days, frequency_orders, orders_last_90d, monetary features. Consistent with retail domain knowledge - if a non-RFM feature dominated, investigate.
- Territory and customer-type features should sit lower.

Train-vs-val gap:

- F1 train $\gg$ F1 val (gap > 0.10) indicates overfitting; tighten

```

min_samples_leaf or cap max_depth.
- val and test should be within ~1-2 percentage points; larger gaps
  suggest val was used too aggressively for tuning.
"""

```

```

In [166]: # Do not modify this code
print_tile(size="h3", key='model_performance_explanations', value=model_performance_explanations)

model_performance_explanations

```

Comparison to baselines: - majority : always predicts 0. F1 / precision / recall on the positive class = 0 by construction. ROC-AUC = 0.5, PR-AUC = positive-class rate. - stratified : random predictions from class prior. F1 $\approx$ positive rate, AUC = 0.5. - rf : the RandomForest above. The RandomForest must beat both baselines on F1 (positive class) AND PR-AUC to be useful. ROC-AUC $\geq$ 0.75 and PR-AUC at least 2x the positive rate would be a good first-deployment result. Confusion matrix interpretation: - TP (bottom-right): customers correctly flagged as 'will reorder'. - FP (top-right) : wrongly flagged — wasted campaign spend. - FN (bottom-left) : missed reorderers — lost retention opportunity. - TN (top-left) : correctly identified non-reorderers. class_weight='balanced' deliberately biases the model toward recall. Feature importances: - Top features expected: recency_days, frequency_orders, orders_last_90d, monetary features. Consistent with retail domain knowledge — if a non-RFM feature dominated, investigate. - Territory and customer-type features should sit lower. Train-vs-val gap: - F1 train F1 val (gap 0.10) indicates overfitting; tighten min_samples_leaf or cap max_depth. - val and test should be within ~1-2 percentage points; larger gaps suggest val was used too aggressively for tuning.

J.5 Business Impact from Current Model Performance

Provide some analysis on the model impacts from the business point of view

```

In [167]: # Translate model performance into business numbers.
# ADJUST these constants to your organisation's actuals before deployment.
COST_PER_CONTACT = 5.0 # USD per marketed customer
AVG_REORDER_VALUE = 250.0 # USD avg sub_total of a reorder
MARGIN_RATE = 0.20 # USD profit per USD revenue
RETENTION_UPLIFT_RATE = 0.15 # share of contacted reorderers whose order is incremental

y_val_proba = model.predict_proba(X_val)[: , 1]
thresholds = np.linspace(0.05, 0.95, 19)
rows = []
for t in thresholds:
    pred = (y_val_proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_val_s, pred, labels=[0, 1]).ravel()
    contacts = tp + fp
    revenue_uplift = tp * AVG_REORDER_VALUE * RETENTION_UPLIFT_RATE
    profit = revenue_uplift * MARGIN_RATE - contacts * COST_PER_CONTACT
    rows.append({
        'threshold': round(t, 2), 'tp': tp, 'fp': fp, 'fn': fn, 'tn': tn,
        'precision': tp / (tp + fp) if (tp + fp) > 0 else 0,
        'recall' : tp / (tp + fn) if (tp + fn) > 0 else 0,
        'contacts' : contacts, 'profit_usd': profit,
    })

biz = pd.DataFrame(rows)
best = biz.loc[biz['profit_usd'].idxmax()]
print(f'Best threshold by profit: {best["threshold"]} '
      f'profit ${best["profit_usd"]:.0f} '
      f'precision {best["precision"]:.2%} recall {best["recall"]:.2%}')
display(biz.style.format({'precision': '{:.2%}', 'recall': '{:.2%}', 'profit_usd': '${:,.0f}'}))

alt.Chart(biz).mark_line(point=True).encode(
    x=alt.X('threshold:Q', title='Decision threshold'),
    y=alt.Y('profit_usd:Q', title='Estimated campaign profit (USD)'),
    tooltip=['threshold', 'precision', 'recall', 'contacts', 'profit_usd']
).properties(width=520, height=260, title='Business profit vs decision threshold (validation set)')

```

Best threshold by profit: 0.9 profit \$298 precision 96.95% recall 37.80%

	threshold	tp	fp	fn	tn	precision	recall	contacts	profit_usd
0	0.050000	334	1212	2	897	21.60%	99.40%	1546	\$-5,225
1	0.100000	330	907	6	1202	26.68%	98.21%	1237	\$-3,710
2	0.150000	325	694	11	1415	31.89%	96.73%	1019	\$-2,658
3	0.200000	315	552	21	1557	36.33%	93.75%	867	\$-1,972
4	0.250000	308	433	28	1676	41.57%	91.67%	741	\$-1,395
5	0.300000	297	365	39	1744	44.86%	88.39%	662	\$-1,082
6	0.350000	287	298	49	1811	49.06%	85.42%	585	\$-772
7	0.400000	277	243	59	1866	53.27%	82.44%	520	\$-522
8	0.450000	256	197	80	1912	56.51%	76.19%	453	\$-345
9	0.500000	246	151	90	1958	61.96%	73.21%	397	\$-140
10	0.550000	226	120	110	1989	65.32%	67.26%	346	\$-35
11	0.600000	203	87	133	2022	70.00%	60.42%	290	\$72
12	0.650000	190	60	146	2049	76.00%	56.55%	250	\$175
13	0.700000	179	45	157	2064	79.91%	53.27%	224	\$222
14	0.750000	172	31	164	2078	84.73%	51.19%	203	\$275
15	0.800000	159	23	177	2086	87.36%	47.32%	182	\$282
16	0.850000	145	16	191	2093	90.06%	43.15%	161	\$282
17	0.900000	127	4	209	2105	96.95%	37.80%	131	\$298
18	0.950000	88	0	248	2109	100.00%	26.19%	88	\$220

Out[167]...

```
In [171]: business_impacts_explanations = """
Cost-of-error model (assumptions explicit so they can be re-set before
deployment):
- Cost per contact ~$5 (offer + email + fulfilment overhead).
- Average reorder revenue ~$250 at ~20% margin.
- Incrementality ~15% – share of contacted reorderers whose order would
  NOT have happened without the campaign.

Threshold tuning:
- The default 0.5 threshold is rarely optimal for imbalanced, cost-
  asymmetric problems. We sweep thresholds and pick the one that
  maximises expected campaign profit on validation.
- Profit curve typically has a clear optimum: too low → contact too many,
  cost outpaces uplift; too high → contact too few, leave revenue on
  the table.
- The chosen threshold is the operational deliverable, the marketing
  team uses it to size each monthly campaign.

Sensitivity:
- Optimum is sensitive to incrementality rate and average reorder value.
  Both should be measured via a randomised holdout experiment in
  production before fixing the threshold.
- The model itself is stable across these assumptions, they only shift
  the threshold, not the customer ranking.

Caveat:
- Profit numbers are illustrative. Real deployment requires finance-
  signed-off unit costs and a measured incrementality rate.
- Natural extension: a two-stage model = this classifier × a regression
  on expected order value, ranked by EXPECTED PROFIT, not propensity.
"""
```

```
In [172]: # Do not modify this code
print_tile(size="h3", key='business_impacts_explanations', value=business_impacts_explanations)
```

Cost-of-error model (assumptions explicit so they can be re-set before deployment): - Cost per contact ~ 5(*offer + email + fulfilment overhead*). – *Average reorder revenue* 250 at ~20% margin. - Incrementality ~15% — share of contacted reorderers whose order would NOT have happened without the campaign. Threshold tuning: - The default 0.5 threshold is rarely optimal for imbalanced, cost-asymmetric problems. We sweep thresholds and pick the one that maximises expected campaign profit on validation. - Profit curve typically has a clear optimum: too low → contact too many, cost outpaces uplift; too high → contact too few, leave revenue on the table. - The chosen threshold is the operational deliverable, the marketing team uses it to size each monthly campaign. Sensitivity: - Optimum is sensitive to incrementality rate and average reorder value. Both should be measured via a randomised holdout experiment in production before fixing the threshold. - The model itself is stable across these assumptions, they only shift the threshold, not the customer ranking. Caveat: - Profit numbers are illustrative. Real deployment requires finance- signed-off unit costs and a measured incrementality rate. - Natural extension: a two-stage model = this classifier × a regression on expected order value, ranked by EXPECTED PROFIT, not propensity.

H. Project Outcomes

```
In [174]: experiment_outcome = "Hypothesis Confirmed"
# - 'Hypothesis Confirmed' : RF beats both baselines on F1 AND
#                           PR-AUC; val/test gap < 2 pp.
# - 'Hypothesis Partially Confirmed' : RF beats stratified baseline but
#                                   overfitting or test drift visible.
# - 'Hypothesis Rejected' : RF F1 ≈ stratified baseline.
#                           Use case / features need rework.

In [175]: # Do not modify this code
print_tile(size="h2", key='experiment_outcomes_explanations', value=experiment_outcome)
```

experiment_outcomes_explanations

Hypothesis Confirmed

```
In [179]: experiment_results_explanations = """
Outcome summary
- The RandomForest classifier demonstrates a learnable signal for
  repeat-purchase prediction over a 90-day horizon. It beats both the
  majority and stratified dummy baselines on F1 and PR-AUC.
- Feature importance is consistent with retail domain knowledge:
  recency_days, frequency_orders, and orders_last_90d are the dominant
  predictors. Monetary features and online_order_ratio provide
  secondary signal.
- A threshold-tuned, cost-aware decision rule turns the raw score into
  a profitable campaign policy (J.5).

New insights gained
- 28% of rows in customer.csv were exact duplicates a critical data-
  quality finding from EDA that the pipeline now handles in E.1.
  Without dedup, customer-level features would be silently corrupted.
- The ~1.5% segment with BOTH person_id AND store_id populated is
  legitimate (sole-trader accounts) and is modelled explicitly via
  two non-mutually-exclusive flags.
- 88% online dominance makes online_order_ratio a useful but not
  dominant feature; the 12% offline minority carries most large-value
  orders and warrants separate per-segment analysis at HD level.

Recommended next steps, ranked by expected uplift
1. Cross-validated hyperparameter search on F1 (~+0.02-0.05 F1, ~2h
  compute). Current values are pragmatically chosen, not exhaustively
  tuned.
2. Gradient-boosting comparison (XGBoost / LightGBM) (~+0.02-0.05 F1).
```

```

GBM family typically edges RF on tabular data.
3. Two-stage model: classifier × regression on expected reorder value;
rank by expected profit, not propensity. Higher campaign ROI without
changing the classifier itself.
4. Rolling-cutoff evaluation to verify temporal stability. If metrics
drift, schedule monthly retraining.
5. SHAP analysis at the per-customer level for stakeholder trust at
deployment time.

Production deployment recommendation
- Monthly batch job: run prep pipeline → score active customer base →
write top-N customers (by score) to marketing-targeting table.
- Monitor: positive-class rate; F1 on previous month's predictions
once labels become observable; realised vs predicted campaign profit.
- Retrain quarterly, or whenever monitored F1 drops by > 0.05.
"""

```

```

In [180]: # Do not modify this code
print_tile(size="h2", key='experiment_results_explanations', value=experiment_results_explanations)

```

Outcome summary - The RandomForest classifier demonstrates a learnable signal for repeat-purchase prediction over a 90-day horizon. It beats both the majority and stratified dummy baselines on F1 and PR-AUC. - Feature importance is consistent with retail domain knowledge: recency_days, frequency_orders, and orders_last_90d are the dominant predictors. Monetary features and online_order_ratio provide secondary signal. - A threshold-tuned, cost-aware decision rule turns the raw score into a profitable campaign policy (J.5). New insights gained - 28% of rows in customer.csv were exact duplicates a critical data-quality finding from EDA that the pipeline now handles in E.1. Without dedup, customer-level features would be silently corrupted. - The ~1.5% segment with BOTH person_id AND store_id populated is legitimate (sole-trader accounts) and is modelled explicitly via two non-mutually-exclusive flags. - 88% online dominance makes online_order_ratio a useful but not dominant feature; the 12% offline minority carries most large-value orders and warrants separate per-segment analysis at HD level. Recommended next steps, ranked by expected uplift 1. Cross-validated hyperparameter search on F1 (~+0.02-0.05 F1, ~2h compute). Current values are pragmatically chosen, not exhaustively tuned. 2. Gradient-boosting comparison (XGBoost / LightGBM) (~+0.02-0.05 F1). GBM family typically edges RF on tabular data. 3. Two-stage model: classifier × regression on expected reorder value; rank by expected profit, not propensity. Higher campaign ROI without changing the classifier itself. 4. Rolling-cutoff evaluation to verify temporal stability. If metrics drift, schedule monthly retraining. 5. SHAP analysis at the per-customer level for stakeholder trust at deployment time. Production deployment recommendation - Monthly batch job: run prep pipeline → score active customer base → write top-N customers (by score) to marketing-targeting table. - Monitor: positive-class rate; F1 on previous month's predictions once labels become observable; realised vs predicted campaign profit. - Retrain quarterly, or whenever monitored F1 drops by 0.05.

```
In [178]: data_transformation_2_explanations
```

```
Out[178]: "\nterritory_id is nominal one-hot encoded so the model doesn't treat\nterritory 5 as 'closer' t
o territory 6 than to territory 1.\n\nVal/test columns are reindexed to the training schema to h
andle the rare\ncase where a territory appears in val/test but not train.\n\nOnly 4 territories
→ 4 additional columns. Minimal dimensional cost.\n"
```

```
In [178]:
```

K. Ethics, Privacy & Indigenous Considerations

This section makes the ethical, privacy, fairness and Indigenous-data considerations of deploying the propensity classifier explicit. It is the notebook-level counterpart to Section 7 of the written report.

```
In [ ]: ethics_full_discussion = """
Ethics, Privacy & Indigenous Considerations

Affected parties:
- Direct: customers in the targeting universe; marketing and sales
  operators who consume the scores.
- Indirect: non-customer household members who share email addresses;
  communities under-represented in the training data (including
  potentially Aboriginal and Torres Strait Islander customers, regional
  and remote customers, recent migrants, low-digital-engagement
  customers); competitors and the broader market.

Privacy (Australian Privacy Act 1988 / APPs):
- Personal information is processed because UUIDs join back to
  identifiable customers. APP 6 (use) and APP 11 (security) apply.
- The pipeline drops raw identifiers before training; the model itself
  is non-PII. The deployed SCORE table, however, is keyed to customer
  and IS personal information.
- Recommendations: plain-English description in privacy notice; self-
  service opt-out at the targeting layer; role-based access on the
  score table; minimum retention period; six-monthly audit.

Indigenous data considerations (AIATSIS Code of Ethics 2020; CARE
principles):
- Collective benefit: monitor that outcomes do not systematically
  advantage non-Indigenous customers via richer feature histories.
- Authority to control: where applicable, data-sharing agreements with
  Indigenous community partners should specify permissible model uses
  and review cadence.
- Responsibility: the operator is responsible for downstream
  consequences of differential targeting, including redress mechanisms.
- Ethics: do not introduce Indigenous-related attributes as predictive
  features without a documented, community-endorsed purpose.

Fairness:
- Engagement-driven feedback loop: contacted customers score higher
  next cycle, creating self-reinforcing inequity. Mitigation: 10%
  randomised holdout per campaign.
- Demographic disparate impact: sparse-history segments score lower
  and receive less outreach, widening the gap. Mitigation: per-segment
  recall monitoring with a 10-percentage-point alert threshold against
  overall recall.

Transparency:
- Global feature importances_ above for stakeholder review.
- Per-customer SHAP values recommended at deployment for APP-compliant
  explanations.

Benefits:
- More efficient retention spend; reduced customer fatigue; recall-
  leaning thresholds aligned with asymmetric LTV costs.

Risks:
- Privacy leakage from joined transactional behaviour to identifiable
  customers; engagement-loop entrenchment of disparities; disparate
  impact on under-represented segments; model staleness as channel
  mix drifts; operator over-reliance on score.
```

Recommendations:

- Roll out in shadow mode for two campaigns before going live.
- Maintain a 10% randomised holdout per campaign to measure incrementality and prevent feedback-loop entrenchment.
- Conduct six-monthly fairness + Indigenous-data-governance audit signed off by a model-risk committee.
- Track per-segment recall monthly; trigger remediation on breach.
- Implement self-service opt-out from automated propensity-based marketing, enforced at the targeting layer.

References (full list in the written report):

- Australian Privacy Act 1988 (Cth) & Australian Privacy Principles.
- AIATSIS Code of Ethics for Aboriginal and Torres Strait Islander Research (2020).
- Carroll et al. (2020), CARE principles for Indigenous data governance, Data Science Journal 19.
- Mehrabi et al. (2021), A survey on bias and fairness in machine learning, ACM Computing Surveys 54(6).

"""

```
print_tile(size="h3", key='ethics_full_discussion', value=ethics_full_discussion)
```